

FinTech Core

Sistema de Gestão e Simulação de Transações Financeiras

Uma aplicação web de gestão financeira pessoal com carteira digital simulada, transferências internas entre usuários e relatórios mensais por categoria — construída do banco de dados ao deploy com foco em lógica, integridade e boas práticas de mercado.

AUTORES

Kaue Alves & Nicolas

INSTITUIÇÃO

FECAP — Ciência da Computação

DURAÇÃO

8 semanas · 5 dias por semana · 1h30 – 2h diárias

STACK PRINCIPAL

Node.js · TypeScript · PostgreSQL · Prisma · React

ÁREAS

Banco de Dados · Backend · Frontend · Deploy

OBJETIVO FINAL

Portfólio público + deploy gratuito + vídeo de demonstração

SUMÁRIO

O que você vai encontrar neste documento

8 semanas

Cronograma dividido em dois meses: fundação técnica e produto final publicado.

2 devs

Papéis separados por afinidade: persistência de dados e regras de negócio.

1h30 – 2h/dia

Ritmo realista de dedicação, de segunda a sexta, com check-in diário de 15 minutos.

- | | | |
|---|---|----|
| 01 | Visão Geral do Projeto | 3 |
| O que é o sistema, o que ele faz na prática e por que ele é estratégico para o currículo. | | |
| 02 | Estratégia contra o Medo do Front-end | 5 |
| Como entregar uma interface profissional sem escrever CSS do zero. | | |
| 03 | Divisão de Papéis e Responsabilidades | 6 |
| Kaue — Arquiteto de Banco de Dados · Nicolas — Engenheiro de Backend. | | |
| 04 | Cronograma Detalhado de 8 Semanas | 7 |
| Semana a semana, com objetivo, estudo e prática de cada integrante. | | |
| 05 | Como se Organizar e Manter o Foco | 11 |
| Check-in diário, quadro de tarefas e disciplina de ramos no GitHub. | | |
| 06 | Resultado Esperado ao Final das Férias | 12 |
| Entregáveis, checklist de portfólio e posicionamento profissional. | | |

Como ler o cronograma

Cada semana traz um **objetivo comum** e, abaixo, duas colunas: a coluna **KAUE** e a coluna **NICOLAS**. Dentro de cada coluna, as tarefas aparecem divididas em Estudo (o que precisa ser entendido antes) e Prática (o que precisa ser construído e entregue).

As semanas 1 a 4 formam o **motor do sistema** — banco, autenticação e movimentações financeiras. As semanas 5 a 8 transformam esse motor em **produto** — relatórios, interface, deploy público e portfólio.

Linha do tempo das 8 semanas

MÊS 1 · O MOTOR DO SISTEMA

1

Ambientação
ferramentas

2

Modelagem
de dados

3

Autenticação
e segurança

4

Transações
ACID

5

Relatórios
e Swagger

6

Interface
React

7

Deploy
na nuvem

8

Portfólio
e LinkedIn

MÊS 2 · PRODUTO E PORTFÓLIO

Mês 1 entrega o motor do sistema; Mês 2 transforma esse motor em produto publicado e portfólio.

Visão Geral: o que é o projeto e por que ele existe

O projeto consiste na criação de um **sistema web de gestão financeira pessoal com simulação de transações de carteira digital**. Na prática, funciona como uma versão simplificada de um aplicativo bancário moderno (como o Nubank), porém focado na organização pessoal de despesas e receitas.

O que o sistema faz na prática

- 1 Cadastro e Carteira.** O usuário cria uma conta e recebe automaticamente uma carteira virtual fictícia com saldo inicial zerado.
- 2 Entradas e Saídas.** O usuário pode registrar depósitos (como seu salário de mentira) e saídas do dia a dia — por exemplo, R\$ 40 em alimentação, R\$ 20 em transporte.
- 3 Transferências Internas (Simulação).** Um usuário cadastrado pode transferir valores virtuais para outro usuário da plataforma. O sistema cuida para que o saldo de quem envia diminua e o de quem recebe aumente na mesma fração de segundo — funcionando exatamente como a troca de ouro ou moedas entre dois jogadores em um videogame, sem envolver bancos reais ou dinheiro de verdade.
- 4 Relatórios e Extrato.** O sistema exibe um extrato das movimentações e um painel com gráficos simples que mostram em que categorias o usuário mais gastou no mês.

Por que esse projeto é estratégico para o currículo

Muitos estudantes criam listas de tarefas simples (To-Do Lists) no início da faculdade. Um sistema financeiro, mesmo simulado, muda o patamar da dupla diante de recrutadores porque exige:

- **Matemática exata.** O dinheiro não pode arredondar de forma errada.
- **Segurança de dados.** Um saldo nunca pode ficar negativo sem permissão e uma transação não pode sumir no meio do caminho.
- **Alinhamento com o mercado de São Paulo.** O setor financeiro (fintechs, bancos, meios de pagamento) é um dos maiores contratantes de estagiários de tecnologia.

As três entidades principais do sistema

Usuários

Nome, e-mail **único** e senha criptografada. É a porta de entrada: sem login válido, nenhuma outra operação acontece.

Contas

Cada usuário recebe uma **carteira virtual** com saldo em tipo decimal. Guarda o valor disponível no momento.

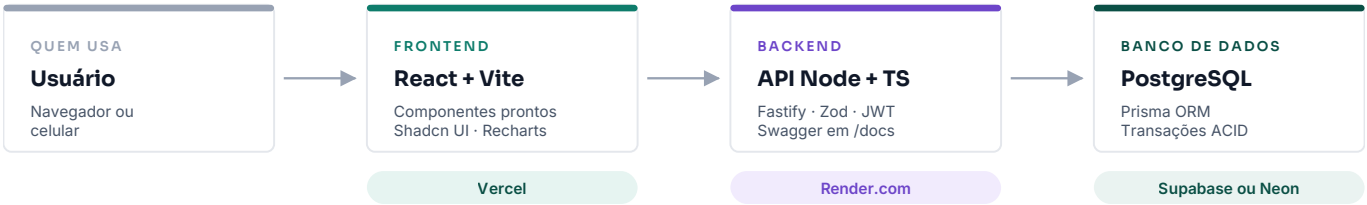
Transações

Histórico de **depósitos, saques e transferências**, com valor, categoria, data e as contas envolvidas.

Relação **1 para N**: um usuário possui uma conta, e essa conta acumula muitas transações. É esse desenho que o Kaue converte no arquivo `schema.prisma` na semana 2.

Tecnologias e Arquitetura do Sistema

O caminho que um clique percorre até virar um registro no banco de dados.



Fluxo de uma operação: a tela em React envia a requisição → a API valida com Zod → o Prisma Client grava no PostgreSQL dentro de uma transação ACID.

Hospedagem em planos gratuitos: Vercel (interface), Render.com (API) e Supabase ou Neon (banco de dados) — sem custo para publicar o projeto.

Tecnologias que serão utilizadas

CAMADA	FERRAMENTAS	RESPONSÁVEL PRINCIPAL
Banco de Dados	PostgreSQL, Prisma ORM, DBeaver, dbdiagram.io	Kaue
Backend / API	Node.js, TypeScript, Fastify ou Express, Zod, JWT, bcryptjs	Nicolas
Documentação	Swagger (interface interativa em <code>/docs</code>)	Nicolas
Frontend	React + Vite, Shadcn UI (componentes prontos), Recharts	Ambos
Infraestrutura	Supabase ou Neon (banco), Render.com (API), Vercel (interface)	Kaue (banco) · Nicolas (API)
Organização	Trello ou Notion, GitHub com ramos separados por área	Ambos

Exemplo de fluxo: uma transferência entre usuários

- O usuário A toca em **"Transferir"** na interface e informa o destinatário e o valor.
- A API recebe a requisição, valida o corpo com **Zod** (valor positivo, formato monetário correto) e identifica o remetente pelo token **JWT**.
- O service abre uma transação no banco (`prisma.$transaction`) antes de qualquer alteração de saldo.
- Dentro da transação: **debita** da conta de A, **credita** na conta de B e **grava** o registro na tabela de Transações.
- Se qualquer etapa falhar, o **ROLLBACK** desfaz tudo — nenhum saldo fica pela metade e nada desaparece no meio do caminho.

A Estratégia contra o Medo do Front-end

Como entregar uma interface limpa e profissional sem desenhar telas nem escrever CSS do zero.

Tanto o Kaue quanto o Nicolas sentem facilidade com raciocínio lógico (vindo da experiência prévia com C#) e sentem atrito com design, CSS e estilização de páginas web.

Regra de ouro do projeto

Ninguém desenhará páginas ou escreverá código CSS do zero.

Como isso se aplica em cada lado do sistema

No Backend

- O foco total será a **qualidade da lógica**, das **regras de negócio** e da **estrutura do banco**.

No Frontend

- A dupla utilizará **bibliotecas de componentes pré-fabricados** (como **Shadcn UI** ou templates prontos em **React**).
- Nesses modelos, os botões, caixas de texto, tabelas e gráficos **já vêm desenhados por designers profissionais**.
- O trabalho da dupla será apenas **conectar as caixas de texto com o sistema** que criaram.

Documentação Automática (Swagger)

- O sistema contará com uma **página técnica que lista todas as operações da aplicação** com botões clicáveis.
- Isso permite **demonstrar o funcionamento do sistema mesmo sem depender de telas complexas**.

Por que isso funciona a favor de vocês

O resultado visual final depende do **componente pronto**, não da habilidade de design da dupla. Isso libera tempo mental para o que realmente diferencia o projeto no currículo: modelagem de dados, integridade transacional e regras de negócio bem escritas — exatamente as competências que uma entrevista técnica de fintech costuma cobrar.

Divisão de Papéis e Responsabilidades

A divisão respeita exatamente o que cada integrante gosta mais de trabalhar, garantindo aprendizado focado e trabalho em equipe estruturado.

K

Kaue

ARQUITETO DE BANCO DE DADOS

Responsabilidade: toda a camada de persistência e integridade da informação.

TAREFAS PRINCIPAIS

- 1 Desenhar a estrutura das tabelas relacionais no **PostgreSQL**.
- 2 Garantir os tipos de dados adequados, evitando perda de precisão em valores monetários.
- 3 Configurar e manter as migrações através do **Prisma ORM**.
- 4 Escrever as consultas de relatórios mensais e o script de geração de dados de teste (seeds).

N

NicolasENGENHEIRO DE BACKEND
E REGRAS DE NEGÓCIO

Responsabilidade: a construção da API, controle de fluxo e segurança da aplicação.

TAREFAS PRINCIPAIS

- 1 Estruturar o servidor em **Node.js com TypeScript**, cuja tipagem é muito próxima do C# que já conhecem.
- 2 Implementar a criptografia de senhas e a emissão de tokens de acesso seguros (JWT).
- 3 Criar os validadores para impedir que dados errados cheguem ao banco (**Zod**).
- 4 Configurar a interface interativa do **Swagger** para testes rápidos da API.

Onde as duas frentes se encontram

O ponto de contato dos dois trabalhos é o arquivo `schema.prisma`: é ali que o desenho do banco feito pelo Kaue vira código que o backend do Nicolas consegue consultar. Sempre que esse contrato mudar, os dois ajustam juntos antes de qualquer implementação nova.

Já na fase de frontend (semana 6), a dupla trabalha em conjunto: Kaue conecta extratos e gráficos, Nicolas conecta formulários e chamadas de API — cada um reaproveitando a parte do sistema que mais conhece.

Cronograma Detalhado de 8 Semanas (2 Meses)

Cada semana tem um objetivo único e tarefas de estudo e prática distribuídas entre os dois integrantes.

MÊS 1 · SEMANAS 1 A 4

Aprendizado, Banco de Dados e o Motor do Sistema

Ambientação das ferramentas, modelagem relacional, autenticação segura e movimentações financeiras com transações ACID.

SEMANA 1

Ambientação, Ferramental e Primeiros Passos

OBJETIVO Ter os programas necessários rodando sem erros nos computadores de ambos.

KAUE · banco de dados

ESTUDO

Tipos primitivos no PostgreSQL. Compreender por que valores monetários devem ser armazenados como `DECIMAL(12, 2)` ou como números inteiros representando centavos, e **nunca como** `FLOAT`.

PRÁTICA

Criar uma conta no serviço de banco em nuvem **Supabase** ou instalar o **PostgreSQL** local. Baixar o programa **DBeaver** para visualizar as tabelas. Criar manualmente via comando SQL uma tabela de teste para entender a sintaxe.

NICOLAS · backend

ESTUDO

Sintaxe básica do **TypeScript** (interfaces, tipos e funções tipadas) e o funcionamento do protocolo HTTP (verbos `GET`, `POST`, códigos de resposta `200`, `400`, `404`).

PRÁTICA

Inicializar um projeto Node.js com TypeScript e subir um servidor simples usando **Fastify** ou **Express** que retorne uma mensagem de sucesso na tela ao ser acessado.

SEMANA 2

Desenho da Estrutura e Ligação Backend-Banco

OBJETIVO Fazer o código do Nicolas conversar diretamente com o banco de dados do Kaue.

KAUE · banco de dados

ESTUDO

Modelagem de Entidade e Relacionamento (1 para N) e introdução à sintaxe do **Prisma ORM**.

PRÁTICA

Desenhar a estrutura do sistema no site gratuito **dbdiagram.io** (Tabela de Usuários, Contas e Transações). Converter esse desenho para o arquivo `schema.prisma`.

NICOLAS · backend

ESTUDO

Arquitetura em camadas simples (Controllers, Services e Rotas) para não deixar o código bagunçado.

PRÁTICA

Integrar o Prisma Client ao projeto Node. Rodar a primeira migração com o Kaue (`npx prisma migrate dev`), garantindo que o código consiga salvar um registro simples no banco.

SEMANA
3

Segurança, Autenticação e Criação de Usuários

OBJETIVO O sistema deve permitir que novos usuários se cadastrem e façam login seguro.

KAUE · banco de dados

ESTUDO

Regras de integridade em tabelas relacionais (`UNIQUE` , `NOT NULL` , índices de busca).

PRÁTICA

Garantir no banco que nenhum usuário se cadastre com o mesmo e-mail duas vezes e associar a criação de um usuário à abertura imediata de uma conta com saldo inicial zero.

NICOLAS · backend

ESTUDO

Criptografia de senhas com a ferramenta `bcryptjs` e proteção de rotas via tokens de sessão **JWT**.

PRÁTICA

Criar a rota de cadastro (que criptografa a senha antes de enviar ao Kaue) e a rota de login (que valida os dados e entrega a chave de acesso ao usuário logado).

SEMANA
4

Operações Financeiras e Integridade dos Dados

OBJETIVO Fazer depósitos, saques fictícios e transferências entre contas funcionarem perfeitamente.

KAUE · banco de dados

ESTUDO

Conceito de **Transações ACID** no banco de dados (`BEGIN` , `COMMIT` , `ROLLBACK`) e sua implementação dentro do Prisma ORM (`prisma.$transaction`).

PRÁTICA

Criar a lógica que garante que numa transferência, se o valor for subtraído da Conta A, ele obrigatoriamente entre na Conta B. Se houver qualquer falha técnica, **nada se altera**.

NICOLAS · backend

ESTUDO

Validação de dados de entrada com a biblioteca **Zod**.

PRÁTICA

Criar as rotas de movimentação de dinheiro. Impedir, por código, que usuários tentem movimentar valores negativos, letras no lugar de números, ou transfiram quantias maiores do que o saldo atual da conta.

Cronograma — Mês 2

MÊS 2 · SEMANAS 5 A 8

Relatórios, Interface Visual, Deploy e Divulgação

Consultas agregadas, documentação interativa, telas construídas com componentes prontos, publicação na internet e construção do portfólio.

SEMANA
5

Consultas Avançadas e Documentação Técnica

OBJETIVO Fazer o sistema gerar resumos matemáticos inteligentes e ter uma documentação visual.

KAUE · banco de dados

ESTUDO

Agrupamento de dados com comandos SQL (`GROUP BY`, `SUM`, `COUNT`) e filtros por data.

PRÁTICA

Escrever a consulta que entrega: "Soma de todos os gastos do usuário no mês atual, divididos por categoria".
Criar um script para injetar automaticamente **50 transações de teste** no banco para validação.

NICOLAS · backend

ESTUDO

Configuração da interface visual de documentação técnica via **Swagger**.

PRÁTICA

Integrar a documentação à API para que qualquer pessoa consiga abrir uma página web (`/docs`) e testar as rotas de depósito, saque e transferência através de botões intuitivos.

SEMANA
6

Interface com o Usuário (Frontend Sem Estresse)

OBJETIVO Montar uma tela limpa e funcional para quem prefere interagir clicando em vez de usar telas técnicas.

TRABALHO CONJUNTO

Criar um projeto em **React com Vite** e instalar um conjunto de componentes prontos (como o **Shadcn UI**).

KAUE · extratos e gráficos

Conectar a tabela pronta do Shadcn para exibir o **extrato do usuário** (valores que entraram em **verde**, valores que saíram em **vermelho**) e posicionar um gráfico simples de gastos usando a biblioteca **Recharts**.

NICOLAS · formulários e API

Criar o formulário simples de **Login** e o formulário modal de **"Nova Movimentação"**, conectando as ações aos links da API construída no mês anterior.

SEMANA
7

Publicação Gratuita na Internet (Deploy)

OBJETIVO Tirar o projeto apenas do computador local e deixá-lo acessível por qualquer celular ou navegador.

KAUE · banco em produção

Configurar o banco de dados PostgreSQL em produção na nuvem (usando serviços gratuitos como **Supabase** ou **Neon**). Rodar as migrações definitivas para subir a estrutura.

NICOLAS · hospedagem

Hospedar a API na nuvem gratuitamente (através do **Render.com**) e hospedar a interface visual na **Vercel**. Configurar as variáveis de ambiente necessárias para a conexão segura entre as partes.

SEMANA
8

Construção do Portfólio, GitHub e LinkedIn

OBJETIVO Apresentar o resultado final de maneira profissional para o mercado de trabalho.

KAUE · dados e README

Exportar o **Diagrama do Banco de Dados (DER)** com boa resolução. Escrever a seção de dados do arquivo `README.md` no GitHub, explicando a escolha do PostgreSQL e o tratamento de segurança com transações ACID.

NICOLAS · backend e onboarding

Detalhar no GitHub o fluxo técnico do Backend, as tecnologias utilizadas e o passo a passo simplificado para qualquer pessoa rodar o projeto.

APRESENTAÇÃO EM DUPLA

Gravar um vídeo demonstrativo de tela de **60 a 90 segundos** mostrando o fluxo: realização do login, registro de uma despesa, atualização do gráfico e consulta ao Swagger.

Publicar o vídeo e os links no **LinkedIn de ambos**, destacando a FECAP, as ferramentas utilizadas e o aprendizado adquirido no período de recesso universitário.

Como se Organizar e Manter o Foco Durante as Férias

A maior armadilha de projetos de férias é a perda de ritmo pela ausência de horários obrigatórios. Para evitar que o projeto seja abandonado, três rituais simples resolvem.

1 - Reunião Diária de 15 Minutos (Check-in)

Marquem um horário fixo (por exemplo, todo dia às 14h ou 19h no Discord ou WhatsApp). Não é uma reunião longa, apenas para responder três perguntas:

- O que eu estudei/fiz ontem?
- O que eu pretendo fazer hoje?
- Tem alguma coisa travando meu progresso?

2 - Quadro de Tarefas Simples (Trello ou Notion)

Mantenham três colunas básicas — **Para Fazer**, **Em Andamento** e **Concluído** — e quebrem os estudos e códigos em tarefas pequenas.

Para Fazer

Em Andamento

Concluído

Exemplo de tarefa bem quebrada

Errado: "Estudar banco de dados todo." → **Certo:** "Pesquisar como funciona o tipo `Decimal` no Postgres."

3 - Divisão de Ramos no GitHub

Nunca alterem o código principal (`main`) diretamente. O Kaue cria uma ramificação para os modelos de dados e o Nicolas para as rotas. Vocês revisam as alterações juntos antes de juntar tudo, simulando o dia a dia de um time profissional.

<code>main</code>	Código estável, sempre funcionando. Só recebe código revisado pelos dois.
<code>feat/dados-*</code>	Ramo do Kaue: schema, migrações, consultas de relatório e seeds.
<code>feat/api-*</code>	Ramo do Nicolas: rotas, autenticação, validadores e documentação Swagger.

Resultado Esperado ao Final das Férias

Seguindo esse planejamento com disciplina moderada — cerca de **1h30 a 2 horas de dedicação por dia, de segunda a sexta** —, ao iniciarem o 3º semestre na FECAP vocês terão em mãos um projeto com arquitetura e maturidade muito acima do padrão de alunos de primeiros semestres de graduação.

Checklist final de entregáveis

- ☐ **Repositório no GitHub bem documentado**
README com a explicação do banco (PostgreSQL + transações ACID), o fluxo do backend e o passo a passo para rodar o projeto.
- ☐ **Diagrama do Banco de Dados (DER) exportado em alta resolução**
Imagem do modelo de Usuários, Contas e Transações anexada ao repositório.
- ☐ **Aplicação publicada e acessível pela internet**
Banco no Supabase/Neon, API no Render.com e interface na Vercel — todos em planos gratuitos.
- ☐ **Documentação interativa no Swagger (/docs)**
Todas as rotas testáveis por botões, servindo como demonstração técnica do sistema.
- ☐ **Vídeo demonstrativo de 60 a 90 segundos**
Fluxo completo: login, registro de uma despesa, atualização do gráfico e consulta ao Swagger.
- ☐ **Publicação no LinkedIn dos dois integrantes**
Vídeo e links do projeto, destacando a FECAP, as ferramentas utilizadas e o aprendizado no recesso universitário.

O QUE ESSE PROJETO PROVA PARA UM RECRUTADOR

Que dois estudantes de Ciência da Computação são capazes de sair do exercício de sala de aula e entregar um sistema completo: modelagem relacional com tipos monetários corretos, autenticação com senhas criptografadas, transferências protegidas por transações ACID, uma API documentada e uma interface publicada na internet. É essa combinação — lógica bem feita, dados íntegros e projeto no ar — que transforma um trabalho de férias em conversa de entrevista.

Kaue Alves

Arquiteto de Banco de Dados · FECAP

Nicolas

Engenheiro de Backend e Regras de Negócio · FECAP